JAVA TECHNOLOGIES

STUDY NOTES

MODULE 1 – JAVA FUNDAMENTALS AND OBJECT-ORIENTED PROGRAMMING

CORE JAVA CONCEPTS

1. Java: Features, Byte Code and Platform Independence

Java is a high-level, class-based, object-oriented programming language designed for developing portable and robust applications. A Java program is normally written as source code, compiled into byte code, and executed by the Java Virtual Machine (JVM). The separation between source code compilation and platform-specific execution is the main reason Java programs can be moved between operating systems with little or no change.

Major features

- Simple: Java has a comparatively clean syntax and removes several difficult low-level facilities.
- Object-oriented: classes and objects provide a structured way to model data and behaviour.
- Platform independent: Java source is compiled to platform-neutral byte code.
- Portable: standard primitive types and the JVM reduce dependence on a particular processor.
- Robust: strong type checking, exception handling and automatic memory management improve reliability.
- Secure: Java uses a managed runtime environment and byte-code verification mechanisms.
- Multithreaded: several independent activities can execute concurrently.
- Distributed: Java provides extensive support for network and distributed application development.
- Architecture-neutral: compiled byte code is not tied to a particular CPU instruction set.
- Dynamic/extensible: classes can be loaded when required during execution.

Byte code

Byte code is the intermediate code produced by the Java compiler. When a file such as Student.java is compiled, the compiler generates Student.class containing byte-code instructions. The byte code is designed for the JVM rather than for one specific processor.

```
Student.java  →  javac Student.java  →  Student.class
             Java source                 Byte code
Student.class → JVM → execution on the target platform
```

Why Java is platform independent

- The programmer writes Java source code.
- The Java compiler converts source code into byte code.
- The byte code is stored in a class file.
- A JVM appropriate to the operating system loads the byte code.
- The JVM executes the same byte code on that operating system.

Thus, Java is commonly described as 'write once, run anywhere', provided a compatible Java runtime is available. The operating-system-specific part is mainly the JVM, while the byte code remains platform neutral.

2. JDK, JRE and JVM

The Java development and execution environment can be understood through three related terms: JDK, JRE and JVM.

- JDK (Java Development Kit): the complete development environment. It contains the tools required to write, compile, debug and run Java programs, together with the runtime environment.
- JRE (Java Runtime Environment): provides the components required to run Java applications, including the JVM and standard runtime libraries.
- JVM (Java Virtual Machine): the execution engine that loads, verifies and executes Java byte code.

Relationship

```
JDK
 └── JRE
      ├── JVM
      └── Java runtime libraries
```

Execution sequence

- Create a Java source file with a .java extension.
- Compile it using the Java compiler.
- The compiler checks syntax and type rules and produces byte code.
- The JVM loads the required class.
- The byte code is verified before execution.
- The JVM executes the instructions using the runtime environment.

The distinction is important in examinations: the JDK is mainly associated with development, the JRE with running Java applications, and the JVM with actual execution of byte code.

3. Primitive Data Types, Variables and Type Casting

Java provides eight primitive data types: byte, short, int, long, float, double, char and boolean. Primitive values directly represent basic data rather than objects.

- byte – small signed integer.
- short – larger integer range than byte.
- int – commonly used integer type.
- long – large integer values.
- float – single-precision floating-point value.
- double – double-precision floating-point value.
- char – a single Unicode character.
- boolean – true or false.

Variables

A variable is a named storage location associated with a particular data type. Variables may be local variables, instance variables or static/class variables.

```
int age = 20;
double average = 82.5;
char grade = 'A';
boolean result = true;
```

Type conversion

Type conversion changes a value from one compatible type to another. Widening conversion generally occurs automatically when a value is moved to a type capable of representing a larger range. Narrowing conversion may lose information and therefore normally requires explicit casting.

```
int n = 25;
double d = n;          // widening

double price = 99.75;
int p = (int) price;   // narrowing; p becomes 99
```

In the second example, the fractional part is discarded. Type casting is therefore not merely a change in notation; it can affect the value stored in the destination variable.

4. Operators in Java

Operators are symbols that perform operations on operands. They are essential in calculations, comparisons and decision-making.

- Arithmetic: +, -, *, /, %
- Relational: ==, !=, >, <, >=, <=
- Logical: &&, ||, !
- Assignment: =, +=, -=, *=, /=, %=
- Unary: ++, --, +, -
- Conditional/ternary: condition ? expression1 : expression2
- Bitwise and shift operators: &, |, ^, ~, <<, >> and related forms

Operator precedence determines the order of evaluation when several operators occur in an expression. Parentheses should be used when the intended order needs to be made explicit.

5. Control Statements and Looping

Control statements determine the flow of execution. Java selection statements are used to make decisions, while loops repeat a block of statements.

Selection

- if: executes a block when a condition is true.
- if-else: chooses between two alternatives.
- else-if ladder: evaluates several conditions.
- switch: selects a block based on an expression's value.

Iteration

- for: suitable when initialization, condition and update are known.
- while: tests the condition before each iteration.
- do-while: executes the body at least once before testing.

Jump/control keywords

- break terminates a loop or switch.
- continue skips the remaining statements of the current iteration.
- return exits a method and may return a value.

```
if (n > 0)
    System.out.println("Positive");
else if (n < 0)
```

```
        System.out.println("Negative");
    else
        System.out.println("Zero");

    for (int i = 1; i <= 10; i++)
        System.out.println(n + " x " + i + " = " + n*i);
```

For a 10-mark answer, explain the purpose of the selected control structure, give its syntax or program structure, explain the condition/update and show the expected result.

6. Arrays and Student-Mark Processing

An array is a fixed-size collection of elements of the same data type. Java arrays use zero-based indexing, so the first element is at index 0. The length field gives the number of elements.

```
int[] marks = {75, 82, 68, 90, 85};
int total = 0;
int highest = marks[0];

for (int i = 0; i < marks.length; i++) {
    total += marks[i];
    if (marks[i] > highest)
        highest = marks[i];
}
double average = (double) total / marks.length;
```

The important logic is initialization, traversal, accumulation and comparison. Type casting before division ensures that the average can contain a decimal value.

Advantages

- Easy storage of related values.
- Simple indexed access.
- Efficient traversal using loops.
- Useful for marks, prices, measurements and other homogeneous data.

Limitation

An ordinary Java array has a fixed length after creation. If a dynamically growing collection is required, collection classes such as ArrayList are generally more suitable.

7. String Operations

String is a Java class used to represent character sequences. Strings are objects and are immutable, meaning that operations that appear to modify a string generally create another String value rather than changing the original object.

- length() – returns number of characters.
- charAt(index) – returns a character.
- equals() – compares string contents.
- compareTo() – performs lexicographic comparison.
- substring() – extracts part of a string.
- toUpperCase()/toLowerCase() – changes letter case in the returned value.
- concat() or + – joins strings.

```
String s = "Java";
System.out.println(s.length());
System.out.println(s.charAt(0));
```

```
System.out.println(s.equals("Java"));
System.out.println(s.substring(1));
```

For reversing a string, the characters may be processed from the last index to the first, or a suitable library technique may be used. For vowel/consonant counting, traverse each character, test whether it is alphabetic, and then check whether it belongs to a,e,i,o,u.

8. Classes, Objects, Methods and Access Specifiers

A class is a blueprint containing data members and methods. An object is an instance of a class. A method defines an operation that an object or class can perform.

```
class Student {
    int rollNo;
    String name;

    void display() {
        System.out.println(rollNo + " " + name);
    }
}

Student s = new Student();
s.rollNo = 101;
s.name = "Anu";
s.display();
```

Access control

- public – accessible wherever the class/member is accessible.
- private – accessible within the declaring class.
- protected – accessible within the same package and through inheritance subject to Java access rules.
- default/package access – available to classes in the same package when no modifier is written.

Access control supports encapsulation. A common design is to keep data private and expose controlled public methods for reading or updating it.

9. this Keyword

The keyword this refers to the current object. Its most common examination use is to distinguish instance variables from parameters with the same name.

```
class Student {
    int mark;

    Student(int mark) {
        this.mark = mark;
    }
}
```

- this.mark identifies the current object's instance variable.
- mark identifies the constructor parameter.
- this may also be used to invoke another constructor or instance method.
- this can be passed as an argument when the current object must be supplied to another method.

10. Static Variables and Static Methods

A static member belongs to the class rather than to each individual object. Therefore, one static variable is shared by all instances of the class.

```
class BankAccount {
    static String bankName = "ABC Bank";
```

```
    String holder;

    static void showBank() {
        System.out.println(bankName);
    }
}
```

Static variables are suitable for common information such as an organization name or an object counter. A static method can be called using the class name. It directly accesses static members and does not have an implicit individual-object context like an instance method.

11. Nested and Inner Classes

A nested class is declared inside another class. A static nested class is associated with the outer class itself, whereas a non-static inner class is associated with an instance of the outer class.

```
class Outer {
    int value = 10;

    class Inner {
        void show() {
            System.out.println(value);
        }
    }
}

Outer o = new Outer();
Outer.Inner i = o.new Inner();
i.show();
```

Inner classes are useful for helper functionality that belongs closely to an enclosing class. They can access members of the enclosing object and can improve logical organization.

OBJECT-ORIENTED FEATURES AND EXCEPTION HANDLING

12. Constructors and Constructor Overloading

A constructor initializes a newly created object. It has the same name as its class and has no return type. Constructors are invoked automatically when an object is created.

- A no-argument/default constructor initializes an object without supplied parameters.
- A parameterized constructor receives values from the caller.
- Constructor overloading means providing multiple constructors with different parameter lists.
- Constructors may use this() to invoke another constructor in the same class.

```
class Student {
    int roll;
    String name;

    Student() {
        roll = 0;
        name = "Unknown";
    }

    Student(int r, String n) {
        roll = r;
        name = n;
    }
}
```

Constructor overloading allows objects to be initialized in different ways while keeping initialization logic associated with the class.

13. Method Overloading

Method overloading is compile-time polymorphism. Several methods in the same class have the same method name but different parameter lists. The difference can be in number, type or order of parameters. Return type alone cannot create an overload.

```
class Area {
    double calculate(double r) { return 3.14*r*r; }
    int calculate(int l, int b) { return l*b; }
    double calculate(double b, double h) { return 0.5*b*h; }
}
```

The compiler examines the method call and selects the compatible parameter list. This is called compile-time method resolution.

14. Inheritance and Its Types

Inheritance permits a subclass to acquire accessible members and behaviour from a superclass. It supports code reuse and hierarchical classification.

- Single inheritance: one subclass derives from one superclass.
- Multilevel inheritance: inheritance occurs through more than one level.
- Hierarchical inheritance: multiple subclasses derive from one superclass.
- Java does not support multiple inheritance of classes; interfaces provide a mechanism for implementing multiple contracts.

```
class Employee {
    void work() {
        System.out.println("Employee works");
    }
}

class Manager extends Employee {
    void manage() {
        System.out.println("Manager manages");
    }
}
```

Inheritance should be used when there is a genuine 'is-a' relationship. The subclass can add new members and can override suitable inherited methods.

15. Method Overriding and Dynamic Method Dispatch

Overriding occurs when a subclass supplies a new implementation for an inherited method with a compatible signature. It supports runtime polymorphism.

```
class Vehicle {
    void display() { System.out.println("Vehicle"); }
}

class Car extends Vehicle {
    @Override
    void display() { System.out.println("Car"); }
}

Vehicle v = new Car();
v.display();     // Car
```

Dynamic method dispatch means the JVM selects the overridden method according to the actual object at runtime. A superclass reference can therefore refer to different subclass objects and invoke different implementations.

16. Abstract Classes

An abstract class is declared using abstract and is intended to serve as a base class. It may contain abstract methods as well as concrete methods, fields and constructors. An abstract method has no implementation in the abstract class.

```
abstract class Shape {
    abstract double area();

    void message() {
        System.out.println("Shape");
    }
}

class Circle extends Shape {
    double r;
    Circle(double r) { this.r = r; }

    double area() {
        return 3.14 * r * r;
    }
}
```

An abstract class cannot normally be instantiated directly. Subclasses provide implementations of the abstract methods. Abstract classes are useful when several related classes share common state or behaviour but must implement certain operations differently.

17. Interfaces

An interface specifies a contract that implementing classes agree to fulfil. A class implements an interface using the implements keyword.

```
interface Payment {
    void pay(double amount);
}

class CreditCard implements Payment {
    public void pay(double amount) {
        System.out.println("Card payment: " + amount);
    }
}

class UPI implements Payment {
    public void pay(double amount) {
        System.out.println("UPI payment: " + amount);
    }
}
```

Interfaces are useful when unrelated classes need to follow a common contract. A class can implement multiple interfaces. In modern Java, interfaces may also contain default and static methods, but the essential examination concept is the contract represented by interface methods and their implementation.

18. Packages, Imports and Sub-packages

A package groups related classes and interfaces under a namespace. Packages improve organization, prevent naming conflicts and contribute to access control.

```
package studentinfo;

public class Student {
```

```
    public void display() {
        System.out.println("Student");
    }
}
```

Another program can import the class:

```
import studentinfo.Student;

class Main {
    public static void main(String[] args) {
        Student s = new Student();
        s.display();
    }
}
```

A sub-package is a separately named package such as college.student or college.staff. It should be treated as a distinct package for access and import purposes.

19. Exception Handling

An exception is an abnormal condition that disrupts the normal flow of execution. Java provides structured exception handling so that errors can be detected and handled without allowing the entire application to fail unexpectedly.

Main keywords

- try – encloses statements that may generate an exception.
- catch – handles a matching exception.
- throw – explicitly throws an exception.
- throws – declares exceptions that a method may pass to its caller.
- finally – contains cleanup code that is normally executed after try/catch processing.

```
try {
    int result = 10 / n;
    System.out.println(result);
}
catch (ArithmeticException e) {
    System.out.println("Division by zero");
}
finally {
    System.out.println("Completed");
}
```

User-defined exceptions can be created by defining a suitable exception class and throwing an object when a validation rule is violated. For example, an age or mark validation program can throw an exception when the supplied value is outside the permitted range.

For examination answers, clearly distinguish throw from throws: throw is used at a particular point to explicitly create/raise an exception, while throws appears in a method declaration to indicate possible exception propagation.

MODULE 2 – THREADS, I/O STREAMS, APPLETS, AWT AND SWING

THREADS, MULTITHREADING AND I/O STREAMS

20. Threads and Multithreading

A thread is a lightweight path of execution within a program. Multithreading allows several tasks to progress concurrently within one application. Examples include downloading data while updating a user interface, processing multiple requests, or performing independent calculations.

Creating a thread by extending Thread

```
class MyThread extends Thread {
    public void run() {
        System.out.println("Task running");
    }
}

MyThread t = new MyThread();
t.start();
```

The run() method contains the task. The start() method creates the execution path and causes run() to be scheduled. Calling run() directly does not create a new thread.

Creating a thread with Runnable

```
class Task implements Runnable {
    public void run() {
        System.out.println("Runnable task");
    }
}

Thread t = new Thread(new Task());
t.start();
```

Runnable separates the task from the Thread object. It is particularly useful when a class already extends another class because Java does not support extending two classes.

21. Thread Life Cycle

A thread passes through states controlled by the JVM and scheduler. The exact Java API names include NEW, RUNNABLE, BLOCKED, WAITING, TIMED_WAITING and TERMINATED.

- NEW – Thread object exists but start() has not been called.
- RUNNABLE – thread is eligible to run; Java's RUNNABLE state covers both ready-to-run and actually executing situations.
- BLOCKED – waiting to acquire a monitor lock.
- WAITING – waiting indefinitely for another thread's action.
- TIMED_WAITING – waiting for a specified period.
- TERMINATED – execution has completed.

A common examination diagram is: NEW → RUNNABLE → execution → TERMINATED, with temporary transitions to BLOCKED/WAITING/TIMED_WAITING when synchronization or waiting occurs.

22. Synchronization and Shared Resources

When two or more threads access shared data, unsynchronized operations may cause race conditions. Synchronization ensures that critical sections are executed in a controlled manner.

```
class Account {
    private int balance = 1000;

    synchronized void deposit(int amount) {
        balance += amount;
    }

    synchronized void withdraw(int amount) {
        if (balance >= amount)
            balance -= amount;
    }
}
```

The synchronized keyword uses the object's monitor lock. Only one thread at a time can execute a synchronized instance method for the same object. This is especially important for bank-account examples involving simultaneous deposits and withdrawals.

Benefits

- Prevents inconsistent updates.
- Protects critical sections.
- Maintains data consistency when shared resources are modified.

Synchronization must be applied carefully because excessive locking can reduce concurrency and poorly designed locking can cause deadlock.

23. I/O Streams and File Handling

Java I/O streams represent the flow of data between a source and a destination. Streams are commonly classified as byte streams and character streams. The question bank emphasizes byte-oriented file streams and data streams.

- FileInputStream – reads bytes from a file.
- FileOutputStream – writes bytes to a file.
- DataInputStream – reads primitive data types from an input stream.
- DataOutputStream – writes primitive data types.
- BufferedInputStream – buffers input to improve efficiency.
- BufferedOutputStream – buffers output to improve efficiency.

```
FileOutputStream out = new FileOutputStream("data.txt");
out.write("Hello Java".getBytes());
out.close();

FileInputStream in = new FileInputStream("data.txt");
int ch;
while ((ch = in.read()) != -1)
    System.out.print((char)ch);
in.close();
```

For a file-copy program, repeatedly read a byte or byte array from the input stream and write it to the output stream until end-of-file is reached. Resources should be closed after use; try-with-resources is the preferred modern form.

24. Data Streams and Buffered Streams

Data streams are useful when primitive Java values must be stored and retrieved in binary form. The order of reading must match the order of writing.

```
DataOutputStream out =
    new DataOutputStream(new FileOutputStream("student.dat"));
out.writeInt(101);
out.writeUTF("Anu");
```

```
out.writeDouble(87.5);
out.close();

DataInputStream in =
    new DataInputStream(new FileInputStream("student.dat"));
int roll = in.readInt();
String name = in.readUTF();
double mark = in.readDouble();
in.close();
```

Buffered streams maintain an in-memory buffer, reducing direct access to the underlying file and potentially improving performance for repeated small operations.

APPLETS, AWT, EVENT HANDLING AND SWING

25. Applet Concept and Life Cycle

The question bank treats the Java applet as a small Java program executed in an applet environment and emphasizes its life-cycle methods. Applets are now a legacy technology and modern browsers do not support them, but the concepts remain part of the supplied syllabus.

- init() – called once for initialization.
- start() – called when execution starts or resumes.
- paint(Graphics g) – used to display graphics/text.
- stop() – called when the applet becomes inactive.
- destroy() – called before removal.

A simple life-cycle explanation can be represented as: load → init() → start() → paint()/active execution → stop() → destroy(). The exact timing of repeated paint calls is controlled by the environment.

26. AWT Components, Containers and Controls

AWT stands for Abstract Window Toolkit. It provides classes for building graphical user interfaces.

- Frame – top-level application window.
- Panel – container for grouping components.
- Label – displays text.
- TextField – accepts one-line text.
- Button – triggers an action.
- Checkbox – represents selectable options.
- Choice/List – supports selection.

A container is a GUI component capable of holding other components. A good registration form may place labels and text fields inside a Panel, which is then placed in a Frame.

27. Layout Managers

A layout manager controls the position and size of components inside a container. It reduces the need to specify fixed coordinates.

- FlowLayout – places components in a sequence and wraps them as necessary.
- BorderLayout – divides a container into North, South, East, West and Center regions.
- GridLayout – arranges components in rows and columns.

The layout should be selected according to the interface. GridLayout is convenient for forms or calculator keypads, while BorderLayout is useful for major regions of a window.

28. Event Handling

Event handling allows a GUI program to respond to user actions. The three important elements are event source, event object and event listener.

- Event source – component generating the event, such as a Button.
- Event object – object carrying information about the action.
- Event listener – object/interface implementation that handles the event.

```
Button b = new Button("Click");
b.addActionListener(e -> {
    System.out.println("Button clicked");
});
```

For a button click, the event source is the button, the event is an ActionEvent and the listener is an ActionListener. Mouse events similarly use mouse event classes and listener interfaces.

29. Adapter Classes

Adapter classes provide empty implementations for listener interfaces that contain several methods. They are useful when only one or two event methods are needed.

For example, instead of implementing every method of a mouse listener, a programmer can extend MouseAdapter and override only mouseClicked(), mousePressed() or another required method. This reduces boilerplate and makes the program easier to understand.

30. Swing and JFC

Swing is a GUI toolkit included in the Java Foundation Classes (JFC). It provides a richer set of GUI components than the basic AWT set.

- JFrame – top-level window.
- JLabel – displays text or images.
- JTextField – accepts one-line input.
- JButton – clickable command button.
- Other Swing components include JCheckBox, JComboBox, JList, JTextArea and many more.

A Swing calculator can be designed using a JFrame, a text field for the display, buttons for digits/operators, and an ActionListener to perform operations. Swing components are generally lightweight and begin with J.

MODULE 3 – JDBC AND DATABASE CONNECTIVITY

JDBC ARCHITECTURE, DRIVERS, CONNECTION AND STATEMENTS

31. JDBC Architecture

JDBC stands for Java Database Connectivity. It provides a standard API through which Java applications communicate with relational databases. The architecture separates application code from database-specific implementation by using JDBC interfaces and drivers.

```
Java Application
       ↓
     JDBC API
       ↓
DriverManager / JDBC Driver
       ↓
     Database
```

- Application – contains the user/business logic.
- JDBC API – provides standard database interfaces.
- DriverManager – manages available drivers and establishes connections.
- JDBC driver – translates JDBC operations into database-specific communication.
- Database – stores and processes data.

The architecture allows a Java application to change database systems with relatively limited changes, provided a suitable driver and compatible SQL/database configuration are used.

32. JDBC Driver and MySQL Connectivity

A JDBC driver is a software component that translates Java JDBC calls into operations understood by a specific database. For MySQL, the commonly used driver is MySQL Connector/J.

A database connection normally requires a JDBC URL, a username and a password. The exact driver-loading approach depends on the Java/JDBC version and driver configuration; modern JDBC drivers can be automatically registered when correctly placed on the classpath.

```
Connection con = DriverManager.getConnection(
    "jdbc:mysql://localhost:3306/college",
    "root",
    "password");
```

The database URL identifies the JDBC protocol, database system, host, port and database name. Connection errors should be handled using SQLException.

33. Steps in JDBC Connectivity

1. Import JDBC classes.
2. Make the JDBC driver available to the application.
3. Obtain a Connection using DriverManager.
4. Create Statement, PreparedStatement or CallableStatement.
5. Execute SQL.
6. Process the ResultSet for query operations.
7. Commit or rollback when transaction control is needed.
8. Close the resources.

A strong essay should explain what happens at each step, not merely list the steps. For example, a Connection represents the active communication session between the Java application and database; a statement object carries SQL to the database; and a ResultSet represents query results.

34. Statement, PreparedStatement and CallableStatement

Statement, PreparedStatement and CallableStatement represent different ways of sending database commands.

- Statement – executes SQL strings directly; suitable for simple static statements.
- PreparedStatement – uses parameter placeholders (?) and is suitable for repeated or parameterized SQL. It also helps prevent SQL injection when parameters are bound rather than concatenated.
- CallableStatement – invokes stored procedures or functions.

```
PreparedStatement ps = con.prepareStatement(
    "INSERT INTO student VALUES (?, ?, ?)");
ps.setInt(1, 101);
ps.setString(2, "Anu");
ps.setDouble(3, 85.5);
ps.executeUpdate();
```

For multiple records, the same PreparedStatement can be reused by changing parameter values and executing repeatedly. CallableStatement is appropriate when database-side stored procedures encapsulate processing.

RESULTSET, LARGE OBJECTS, METADATA, TRANSACTIONS AND ERRORS

35. ResultSet and ResultSet Types

A ResultSet represents the rows returned by a SQL SELECT query. The cursor initially appears before the first row, and next() moves it to the next row.

```
ResultSet rs = st.executeQuery("SELECT * FROM student");
while (rs.next()) {
    System.out.println(rs.getInt("rollno"));
    System.out.println(rs.getString("name"));
}
```

Scrollable ResultSet configurations can support navigation such as first(), last(), next() and previous(). The statement can be created with an appropriate result-set type and concurrency mode.

```
Statement st = con.createStatement(
    ResultSet.TYPE_SCROLL_INSENSITIVE,
    ResultSet.CONCUR_READ_ONLY);
```

Concurrency describes whether the ResultSet can be updated. Updatable ResultSets require suitable configuration and a database/query that supports the requested update operations.

36. BLOB and CLOB

BLOB stands for Binary Large Object and stores large binary data such as images and PDF files. CLOB stands for Character Large Object and stores large character data such as lengthy reports or documents.

- BLOB → binary information.
- CLOB → character/text information.

A BLOB can be populated from a binary input stream and retrieved through a binary stream. A CLOB can be handled using character-oriented operations. In an exam answer, explain the data type, typical application, insertion idea and retrieval idea.

37. Database Metadata and ResultSet Metadata

Database metadata describes the database system itself. JDBC provides DatabaseMetaData to obtain information such as database product name, version, supported features and available tables.

```
DatabaseMetaData md = con.getMetaData();
System.out.println(md.getDatabaseProductName());
System.out.println(md.getDatabaseProductVersion());
```

ResultSetMetaData describes the columns in a query result. It is useful when the structure is not known in advance.

```
ResultSetMetaData m = rs.getMetaData();
int n = m.getColumnCount();
for (int i=1; i<=n; i++) {
    System.out.println(m.getColumnName(i));
    System.out.println(m.getColumnTypeName(i));
}
```

A key distinction: DatabaseMetaData describes the database and its capabilities, whereas ResultSetMetaData describes the columns of a particular query result.

38. JDBC Transactions and ACID

A transaction is a logical group of database operations that should be treated as one unit. The four ACID properties are Atomicity, Consistency, Isolation and Durability.

- Atomicity – all required operations succeed as a unit or the transaction is undone.
- Consistency – database rules and constraints remain valid.
- Isolation – concurrent transactions should not improperly affect one another.
- Durability – committed changes remain stored even after subsequent failures.

```
con.setAutoCommit(false);

try {
    // debit account A
    // credit account B
    con.commit();
} catch (SQLException e) {
    con.rollback();
}
```

A bank transfer illustrates the need for transactions. If the debit succeeds but the credit fails, rollback can prevent the database from retaining only half of the intended operation.

39. JDBC Error Handling

Database applications can encounter invalid SQL, connection failures, authentication problems, constraint violations and other database errors. SQLException is the main exception type used for JDBC errors.

```
try {
    Connection con =
        DriverManager.getConnection(url, user, password);
    // SQL operations
} catch (SQLException e) {
    System.out.println("Database error: " + e.getMessage());
}
```

For robust programs, resources should be closed even when an exception occurs. Try-with-resources is a useful modern mechanism because it automatically closes AutoCloseable JDBC resources. When a transaction is active, a failure should normally be followed by rollback.

MODULE 4 – MVC, SERVLETS AND JSP

MVC ARCHITECTURE AND JAVA SERVLETS

40. MVC Architecture

MVC stands for Model–View–Controller. It separates data/business processing from presentation and request handling. This separation makes web applications easier to maintain and extend.

- Model – represents application data and business logic. It may communicate with a database.
- View – presents information to the user, such as an HTML/JSP page.
- Controller – receives the request, invokes suitable processing and chooses the response/view.

Example: in an online student registration system, the View contains the registration form, the Controller receives the submitted details, and the Model validates/stores the student data. The Controller then selects a suitable response.

```
User → View/Form → Controller → Model/Database
                                    ↓
                                  View
                                    ↓
                                  User
```

41. Servlet, Servlet Interface, GenericServlet and HttpServlet

A servlet is a server-side Java component that processes requests and generates responses. A web container manages servlet creation, life cycle and request processing.

- Servlet interface – defines the basic servlet life-cycle contract.
- GenericServlet – protocol-independent convenience base class.
- HttpServlet – specialized base class for HTTP applications and provides methods such as doGet() and doPost().

For normal web applications using HTTP, HttpServlet is the common choice because it directly supports HTTP request methods and response handling.

42. Servlet Life Cycle

The servlet life cycle is managed by the web container.

- The container loads the servlet class and creates the servlet object.
- init() is called once for initialization.
- service() handles incoming requests. In HttpServlet, it dispatches HTTP requests to methods such as doGet() or doPost().
- The servlet may process many requests during its lifetime.
- destroy() is called when the servlet is removed from service.

```
init()
   ↓
service() → doGet()/doPost()
   ↓
service() → doGet()/doPost()
   ↓
destroy()
```

Initialization should contain setup work, while request-specific work belongs in the appropriate request-processing methods. Cleanup resources that belong to the servlet can be performed during destruction.

43. GET, POST and Servlet Response

HTTP GET and POST are two common request methods. doGet() handles GET requests, while doPost() handles POST requests.

```
protected void doPost(HttpServletRequest request,
                      HttpServletResponse response)
                      throws ServletException, IOException {
    String name = request.getParameter("name");
    response.setContentType("text/html");
    response.getWriter().println(
        "<h2>Welcome " + name + "</h2>");
}
```

- GET is commonly used to retrieve information and can expose parameters in the URL.
- POST is commonly used to submit data for processing.
- Neither method by itself provides security; sensitive applications still require HTTPS, authentication, authorization and input validation.
- HttpServletRequest supplies information about the incoming request.
- HttpServletResponse is used to construct the response.

44. Cookies and Session Tracking

HTTP is stateless, so a web application needs a mechanism to recognize or retain information about a user across requests. Session tracking provides this continuity.

A cookie is a small piece of data associated with a website/application and stored by the client. The server can create a cookie and add it to the response.

```
Cookie c = new Cookie("language", "English");
response.addCookie(c);
```

On later requests, the application can inspect received cookies. Session tracking may also use server-side sessions. In examination answers, explain the problem of statelessness, the mechanism used, how information is stored/retrieved, and one suitable application such as an online shopping cart.

45. Servlet Chaining

Servlet chaining means passing processing from one servlet to another. It is useful when a web application divides a task into multiple stages.

One servlet can perform validation or preliminary processing, store information as request attributes, and forward the request to another servlet or resource using a request dispatcher. The receiving component continues the processing and generates the final response.

Typical flow

```
Client
  ↓
Servlet A
  ↓  forward
Servlet B
  ↓
Response
```

Chaining can promote separation of responsibilities, but the application should be designed so that request flow remains understandable.

46. Servlet Error Handling and Security

Servlet error handling aims to provide controlled responses when an exception or invalid request occurs. Java exception handling can be combined with web-container error handling to display suitable messages or error pages.

- Validate input before processing.
- Catch appropriate exceptions rather than hiding all errors.
- Return suitable error responses.
- Do not expose stack traces or sensitive implementation details to users.
- Use authentication and authorization where required.
- Protect sessions and cookies.
- Use HTTPS for sensitive communication.
- Use parameterized SQL/JDBC techniques for database operations.

JAVA SERVER PAGES (JSP)

47. JSP and Its Purpose

Java Server Pages (JSP) is a server-side technology for generating dynamic web content. A JSP page is translated into a servlet by the JSP container and then processed to produce a response.

- Supports dynamic page generation.
- Provides JSP expressions, declarations and directives.
- Provides implicit objects for request/response processing.
- Supports Expression Language (EL).
- Supports standard and custom tag libraries.
- Can be used as a presentation layer in MVC-style web applications.

JSP is often used for presentation, while servlets/controllers and model classes handle request processing and application logic. This separation avoids placing all application logic directly inside a page.

48. JSP Life Cycle

The JSP life cycle describes how a JSP page becomes an executable server-side component.

- Translation – JSP source is translated into servlet source.
- Compilation – generated servlet source is compiled.
- Loading/initialization – the generated servlet is loaded and initialized.
- Request processing/service – the generated servlet processes requests and creates output.
- Destruction – the generated servlet is taken out of service and resources are released.

The key idea is that a JSP page is not simply sent unchanged to the browser. It is processed on the server, and the generated response is returned to the client.

49. JSP Expressions and Declarations

A JSP expression evaluates an expression and places its result into the response. Traditional syntax is:

```
<%= expression %>
```

```
<h2>Name: <%= name %></h2>
<p>Mark: <%= mark %></p>
```

A JSP declaration defines fields or methods that become members of the generated servlet class.

```
<%!
    int square(int n) {
        return n * n;
```

```
    }
%>
```

The method can subsequently be invoked from suitable JSP code. For a 10-mark answer, distinguish expressions (display/evaluate a value) from declarations (define class-level members).

50. JSP Directives

Directives provide instructions to the JSP container and affect page translation/processing. The page directive is particularly important in the supplied question bank.

```
<%@ page contentType="text/html"
    import="java.util.*" %>
```

A page directive can specify attributes such as content type and imported packages. Directives are instructions to the container rather than ordinary HTML content.

51. JSP Implicit Objects

JSP provides predefined objects that can be used without explicit declaration. These objects simplify access to request information, response handling, session data and application context.

- request – current client request.
- response – response sent to the client.
- session – information associated with the current user session.
- out – output writer used to generate response content.
- application – application-wide context.
- config – servlet configuration.
- pageContext – provides access to page/request/session/application information.
- page – reference to the generated servlet object.
- exception – available in appropriate error pages.

```
<%
String name = request.getParameter("name");
session.setAttribute("user", name);
%>
```

52. JSP Action Tags

Standard JSP action tags perform operations during request processing. The supplied question bank emphasizes jsp:include and jsp:forward.

```
<jsp:include page="header.jsp" />
<jsp:forward page="login.jsp" />
```

- jsp:include includes the output of another resource.
- jsp:forward transfers request processing to another resource.
- Action tags operate during request processing, unlike directives which primarily influence translation.

53. Expression Language (EL)

Expression Language provides concise access to attributes and properties in JSP pages. It reduces the need to write Java expressions directly in presentation pages.

```
${student.name}
${requestScope.message}
${sessionScope.user}
```

EL can access values in request, session, application and other scopes. It is particularly useful when a servlet/controller places an object into an attribute and the JSP displays its properties.

54. JSTL and Standard Tag Libraries

JSTL, the JavaServer Pages Standard Tag Library, provides reusable tags for common presentation tasks such as iteration, conditional processing and formatting. It helps reduce Java code embedded directly in JSP pages.

```
<c:forEach var="item" items="${items}">
    <p>${item}</p>
</c:forEach>
```

A JSP using JSTL must have the appropriate tag library available and declared. The exact configuration depends on the JSTL version used by the application.

55. Custom JSP Tags

Custom tags allow developers to package reusable presentation behaviour into a tag that can be used repeatedly across JSP pages. A welcome-message tag is a simple example.

- Create the tag implementation or handler.
- Define/register the tag library using the mechanism required by the JSP environment.
- Make the tag library available to the web application.
- Declare the tag library in the JSP page.
- Use the custom tag where required.

Custom tags promote reuse, improve readability and reduce repeated presentation logic. In a 10-mark answer, explain the purpose, development steps, declaration and an example of use.

ANSWER WRITING PATTERN

A strong long answer should normally contain several of the following elements rather than only a definition.

- Introduction/definition – clearly identify the concept.
- Core explanation – describe what it does and why it is used.
- Components/types – list and explain the important categories.
- Working/architecture/life cycle – provide a sequence or diagram when applicable.
- Syntax or program structure – include a relevant Java code example when the topic is programming-oriented.
- Explanation of the code – identify the important statements and their purpose.
- Advantages/features – mention relevant benefits.
- Application/example – connect the concept with a realistic use case.
- Comparison/distinction – where related concepts are easily confused, clearly differentiate them.
- Conclusion – summarize the main idea in two or three sentences.

High-Priority Comparisons to Learn

- JDK vs JRE vs JVM
- Widening conversion vs narrowing casting
- Class vs object
- Instance member vs static member
- Constructor overloading vs method overloading
- Overloading vs overriding
- Inheritance vs interface implementation
- Abstract class vs interface
- throw vs throws
- Thread vs Runnable
- File streams vs buffered streams vs data streams
- AWT vs Swing
- Event source vs event object vs event listener
- Statement vs PreparedStatement vs CallableStatement
- DatabaseMetaData vs ResultSetMetaData
- BLOB vs CLOB
- commit() vs rollback()
- GenericServlet vs HttpServlet
- GET vs POST
- Cookie vs session tracking
- JSP expression vs declaration vs directive
- JSP action tag vs directive
- EL vs Java code in a JSP page
- JSTL vs custom tag library

Important Program Templates to Practise

- Average and type casting
- Positive/negative/zero and prime checking
- Multiplication table and loop processing
- Array total, average and highest mark
- String reverse and vowel/consonant counting

- Student class with objects and constructors
- this keyword and static members
- Method overloading
- Inheritance and method overriding
- Dynamic method dispatch
- Abstract Shape with Circle/Rectangle
- Payment interface with multiple implementations
- Package and sub-package access
- try-catch, throw, throws and finally
- Thread creation using Thread
- Thread creation using Runnable
- Synchronization with a shared bank account
- File copy using FileInputStream/FileOutputStream
- DataInputStream/DataOutputStream student records
- Buffered input/output
- AWT registration form and event handling
- Swing calculator
- JDBC MySQL connection
- Statement insertion/retrieval
- PreparedStatement update/insertion
- CallableStatement stored procedure
- ResultSet navigation
- BLOB/CLOB storage concept
- Metadata retrieval
- Transaction with commit/rollback
- Servlet form processing
- Servlet life cycle and GET/POST
- Cookies/session tracking
- Servlet chaining
- JSP expression/declaration/directive
- JSP implicit objects
- EL, JSTL and custom tags